



《AI大模型Ragent项目》——三个通道返回30条结果，最终只给模型5条

来自： 拿个offer-开源&项目实战



马丁

2026年05月14日 21:56

开篇引言

上一篇拆解了多通道并行检索的内部实现——SearchChannel 接口抽象、两级线程池隔离、AbstractParallelRetriever 模板方法模式、CompletableFuture 并行调度与容错。读者已经知道怎么把一个用户问题同时丢给多个通道、怎么让通道之间互不阻塞、怎么在某个通道挂掉时返回空结果而不拖垮全局。

文章最后留了一个问题：定向检索从两个 Collection 搜回了 14 条，全局检索从 5 个 Collection 搜回了 15 条，加起来 29 条 Chunk——能直接塞进 LLM 的 Prompt 吗？

V1 版本 Ragent AI 代码未接入关键词检索，但是文章预留了关键词检索通道进行讲解。

答案是不能。把场景拉回电商客服。用户问了一句“AirPods Pro 怎么退货？”，经过意图识别和多通道并行检索后，三个通道各自返回了结果：

意图定向通道 (INTENT_DIRECTED)：命中 3C 数码退货政策 Collection，返回 10 条 Chunk，分数范围 0.82~0.45

关键词检索通道 (KEYWORD_ES)：用 BM25 在全局搜到包含 AirPods、退货关键词的 8 条 Chunk，分数范围 12.5~3.2

全局向量检索通道 (VECTOR_GLOBAL)：在所有 Collection 中做向量检索，返回 10 条 Chunk，分数范围 0.78~0.35

三个通道合计 28 条 Chunk。这 28 条有三个问题：

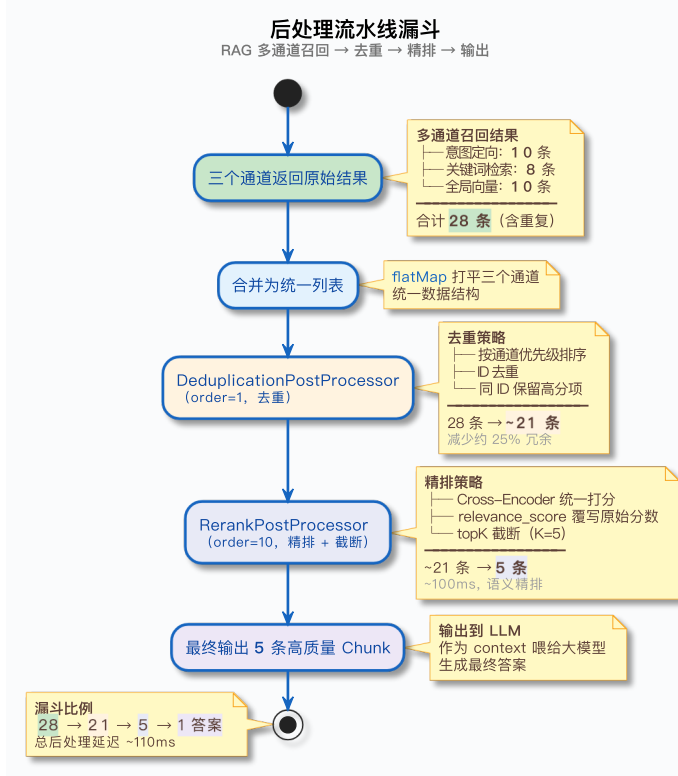
1. **有重复**：同一篇退货政策文档在意图定向和全局向量两个通道都被捞到了，不去重就会在 Prompt 里出现两次，白白浪费 Token
2. **分数不可比**：向量通道返回的分数是余弦相似度 (0~1)，ES 通道返回的是 BM25 分数（理论上无上界），意图定向的 0.82 和 ES 的 12.5 放在一起排序毫无意义
3. **数量太多**：28 条 Chunk 的文本量可能上万 Token，全塞进 Prompt 会挤压生成空间，核心信息被大量低相关内容稀释

后处理流水线要解决的就是这三个问题：**去重** → **精排** → **截断**，最终只留下最相关的 topK 条喂给大模型。

后处理流水线全景

1. 一张图看懂漏斗

先用一张图看看后处理流水线的整体结构。三个通道的原始结果经过两个处理器，逐步收窄到最终 5 条：



漏斗的逻辑很直观：28 条原始 Chunk 经过去重缩减到约 21 条，再经过 Cross-Encoder 精排只保留最相关的 5 条。从数量上看是 28 → 21 → 5，从质量上看是粗糙的多源结果 → 无重复的候选集 → 精选的高质量上下文。

2. 两阶段入口：retrieveKnowledgeChannels

后处理流水线并不是独立运行的，它是 MultiChannelRetrievalEngine 两阶段架构的第二阶段。来看入口方法：

```
@RagTraceNode(name = "multi-channel-retrieval", type = "RETRIEVE_CHANNEL")
public List<RetrievedChunk> retrieveKnowledgeChannels(List<SubQuestionIntent> subIntents, int topK)
    // 构建检索上下文
    SearchContext context = buildSearchContext(subIntents, topK);

    // 【阶段1：多通道并行检索】
    List<SearchChannelResult> channelResults = executeSearchChannels(context);
    if (CollUtil.isEmpty(channelResults)) {
        return List.of();
    }

    // 【阶段2：后置处理器链】
    return executePostProcessors(channelResults, context);
}
```

两阶段的职责很清楚：

- 阶段 1：executeSearchChannels()——并行执行所有启用的 Channel，收集每个通道的原始结果（第 10 篇已经讲过）
- 阶段 2：executePostProcessors()——串行执行后处理器链，做去重、精排、截断（本篇的重点）

阶段 1 的输出 List<SearchChannelResult> 就是阶段 2 的输入。每个 SearchChannelResult 封装了一个通道的检索结果，包含通道类型、通道名称、Chunk 列表和耗时：

```
@Data
@Builder
public class SearchChannelResult {
    private SearchChannelType channelType; // 通道类型 (INTENT_DIRECTED / KEYWORD_ES / VECTOR_GLOBA
    private String channelName;           // 通道名称 (用于日志)
    private List<RetrievedChunk> chunks;  // 检索到的 Chunk 列表
    private long latencyMs;                // 检索耗时 (毫秒)
}
```

后处理器接口：SearchResultPostProcessor

1. 四个方法各管什么

后处理流水线的核心抽象是 SearchResultPostProcessor 接口，它定义了每个处理器必须实现的四个方法：

```
public interface SearchResultPostProcessor {

    /** 处理器名称 */
    String getName();

    /** 处理器优先级（数字越小越先执行） */
    int getOrder();

    /** 是否启用该处理器 */
    boolean isEnabled(SearchContext context);

    /** 处理检索结果 */
    List<RetrievedChunk> process(List<RetrievedChunk> chunks,
                                List<SearchChannelResult> results,
```

```
        SearchContext context);  
  
    }
```

四个方法各司其职：

方法	职责	举例
getName()	返回处理器名称，用于日志和监控	Deduplication、Rerank
getOrder()	返回执行优先级，数字越小越先执行	去重返回 1，精排返回 10
isEnabled()	根据上下文决定是否启用该处理器	当前两个处理器始终返回 true
process()	核心处理逻辑，输入 Chunk 列表，输出处理后的 Chunk 列表	去重、精排 + 截断

2. process() 的入参设计

process() 方法同时接收三个参数，这个设计值得注意：

chunks：上一个处理器的输出，是流水线的传递对象。第一个处理器拿到的是所有通道的 Chunk 合并后的列表，后续处理器拿到的是前一个处理器处理过的列表

results：原始的多通道检索结果，只读引用。去重处理器需要按通道分组处理（先处理高优先级通道的结果），所以需要访问原始的通道结构，而不能只看打平后的 chunks

context：全局检索上下文，携带 topK、用户问题等参数。精排处理器需要 context.getMainQuestion() 作为 Rerank 的 query，需要 context.getTopK() 决定截断数量

打个比方：chunks 是流水线上正在加工的半成品，results 是工艺卡片（记录了原料来自哪个车间），context 是生产工单（写着最终要交付多少件）。不是每道工序都需要看工艺卡片，但去重工序需要——因为它要根据原料来源（通道优先级）决定保留哪一份。

3. 返回值设计：输入一个列表，输出一个列表

每个处理器的返回值是一个新的 List<RetrievedChunk>。不修改输入列表，返回新列表——和 Java Stream 的 .filter() 或 .map() 思路一样。上一个处理器的输出直接变成下一个处理器的输入，链式传递自然成立。

第一步：去重——DeduplicationPostProcessor

1. 为什么去重是第一步

去重处理器的 getOrder() 返回 1，是流水线中最先执行的。原因很直接：如果不去重就精排，同一条 Chunk 在意图定向和全局向量两个通道各出现一次，Rerank 模型要给它打两次分——浪费 API 调用和算力，而且重复 Chunk 还会占据 topK 的名额。

2. 通道优先级排序

2.1 三个通道的优先级

去重不是简单地遍历所有 Chunk 然后去掉重复的。它先按通道优先级排序，再依次处理每个通道的结果：

```
private int getChannelPriority(SearchChannelType type) {  
    return switch (type) {  
        case INTENT_DIRECTED -> 1;    // 意图检索优先级最高  
        case KEYWORD_ES -> 2;         // 关键词检索次之  
        case VECTOR_GLOBAL -> 3;      // 全局检索最低  
        default -> 99;  
    };  
}
```

优先级的设计逻辑：

通道类型	优先级	理由
INTENT_DIRECTED	1（最高）	根据意图精准定位到特定 Collection，命中最靠谱
KEYWORD_ES	2	关键词精确匹配，召回精度高

2.2 优先级影响什么

先处理高优先级通道的结果，后处理低优先级通道的结果。用 `LinkedHashMap` 维护插入顺序——高优先级通道的 `Chunk` 先占坑，低优先级通道遇到同一个 `Chunk` 时，只有分数更高才能替换。

来看完整的去重处理器代码：

```
@Slf4j
@Component
public class DeduplicationPostProcessor implements SearchResultPostProcessor {

    @Override
    public String getName() {
        return "Deduplication";
    }

    @Override
    public int getOrder() {
        return 1; // 最先执行
    }

    @Override
    public boolean isEnabled(SearchContext context) {
        return true; // 始终启用
    }

    @Override
    public List<RetrievedChunk> process(List<RetrievedChunk> chunks,
                                       List<SearchChannelResult> results,
                                       SearchContext context) {
        // 使用 LinkedHashMap 保持顺序并去重
        Map<String, RetrievedChunk> chunkMap = new LinkedHashMap<>();

        // 按通道优先级排序（优先级高的通道结果优先保留）
        results.stream()
            .sorted((r1, r2) -> Integer.compare(
                getChannelPriority(r1.getChannelType()),
                getChannelPriority(r2.getChannelType())
            ))
            .forEach(result -> {
                for (RetrievedChunk chunk : result.getChunks()) {
                    String key = generateChunkKey(chunk);

                    if (!chunkMap.containsKey(key)) {
                        // 新 Chunk, 直接添加
                        chunkMap.put(key, chunk);
                    } else {
                        // 已存在, 合并分数（取最高分）
                        RetrievedChunk existing = chunkMap.get(key);
                        if (chunk.getScore() > existing.getScore()) {
                            chunkMap.put(key, chunk);
                        }
                    }
                }
            });

        return new ArrayList<>(chunkMap.values());
    }

    // ... getChannelPriority() 见上文
}
```

注意这里有一个细节：去重处理器没有用 `chunks` 入参（打平后的列表），而是直接从 `results`（按通道分组的原始结果）入手。因为去重需要按通道优先级处理，而 `chunks` 已经丢失了通道归属信息。这就是 `process()` 方法同时传 `chunks` 和 `results` 两个参数的原因——不同处理器需要不同的数据视图。

3. 去重键生成：generateChunkKey()

3.1 两级策略

去重的关键在于怎么判断两条 Chunk 是同一条。generateChunkKey() 用了两级策略：

```
private String generateChunkKey(RetrievedChunk chunk) {
    return chunk.getId() != null
        ? chunk.getId()
        : String.valueOf(chunk.getText().hashCode());
}
```

- 有 ID 用 ID：Milvus 的 primary key 是唯一的，同一篇文档在不同通道检索出来的 ID 相同。这是最可靠的去重依据
- 无 ID 用内容哈希：text.hashCode() 作为兜底。理论上不会走到这条路径（正常检索结果都有 ID），但防御式编程总是好习惯

3.2 为什么 ID 比内容哈希更可靠

举个例子：同一条 Chunk 在意图定向通道返回时 text 末尾可能多了一个空格（不同通道的文本后处理逻辑可能有差异），导致 hashCode() 值不同，按内容哈希就识别不出是同一条。但 ID 是 Milvus 的主键，不管从哪个通道检索出来都一样。ID 去重是精确匹配，内容哈希只是近似去重的最后防线。

4. 重复时保留高分

当同一个 Chunk 在多个通道都出现时，保留分数更高的那一份：

```
if (chunk.getScore() > existing.getScore()) {
    chunkMap.put(key, chunk);
}
```

用电商场景的具体数字来说明：

- 退货政策文档 chunk-42 在意图定向通道得分 0.82，在全局向量通道得分 0.68
- 意图定向先处理（优先级高），chunk-42 以 0.82 分进入 LinkedHashMap
- 全局向量后处理，遇到 chunk-42，0.68 < 0.82，不替换
- 最终 chunk-42 保留的是 0.82 分

还有一种边界情况：如果全局向量通道对某个 Chunk 的打分反而更高（比如 0.90 > 0.82），也会替换。保留高分的逻辑不受通道优先级影响——优先级只决定谁先占坑，高分逻辑决定最终保留谁的分数。两个机制各管各的。

5. 去重效果演算

用电商场景的 28 条 Chunk 走一遍去重流程：

步骤	处理通道	通道 Chunk 数	新增	重复（保留高分）	重复（分数不变）	Map 总数
1	意图定向（优先级 1）	10	10	0	0	10
2	关键词检索（优先级 2）	8	5	1	2	15
3	全局向量（优先级 3）	10	6	0	4	21

28 条经过去重后变成了 21 条——7 条重复被合并掉了。意图定向作为最高优先级先把 10 条全部放进去；关键词检索的 8 条中有 3 条和意图定向重复（其中 1 条分数更高触发替换，2 条分数更低保持不变），新增 5 条；全局向量的 10 条中有 4 条和前面重复（全部分数更低），新增 6 条。

6. 为什么按优先级去重？

如果实际场景中：

下游不关心 Chunk 的顺序

不同通道返回的同一个 Chunk 对象除了 score 外没有任何区别

那排序确实没有实际意义，因为去重逻辑已经用 取最高分 来处理了，不管哪个通道先遍历，最终结果的 score 都是一样的。排序的核心价值在于控制最终结果的顺序（高优先级通道的结果靠前），而不是去重本身。如果后续有 TopK 截断，这个顺序就很重要。整体来说，是为后续扩展留了个口子。

第二步：精排——RerankPostProcessor

1. 为什么去重后还需要精排

去重后还剩 21 条 Chunk，但来自不同通道，分数标准完全不同：

通道	分数含义	分数范围	举例
意图定向	余弦相似度	0~1	0.82
关键词检索	BM25 分数	0~无上界	12.5
全局向量	余弦相似度	0~1	0.78

意图定向的 0.82 和 ES 的 12.5 放在一起排序没有任何意义——一个是相似度百分比，一个是词频权重的加权和，量纲都不一样。直接按分数排序可能把一条 BM25 分数高但语义不相关的 Chunk 排到最前面。需要一个统一的打分标准，让所有 Chunk 在同一个尺度上重新排序——这就是 Rerank 模型要做的事。

2. Cross-Encoder 简要回顾

咱们在 RAG 基础系列检索策略已经了解过 Bi-Encoder 和 Cross-Encoder 的区别，这里从工程视角简要回顾。

特性	Bi-Encoder（检索阶段）	Cross-Encoder（精排阶段）
输入方式	query 和 chunk 分别编码成向量，比较向量距离	query 和 chunk 拼成一个序列一起送入模型
语义交互	无直接交互，靠向量距离近似	完整的注意力交互，精确理解语义关系
速度	快（chunk 可以预先编码，检索时只需算距离）	慢（每个 query-chunk 对都要单独过一遍模型）
精度	够用但不够精	高
适用场景	从百万文档中粗筛候选	对几十条候选做精排

两者的关系就是经典的粗排 + 精排两阶段架构：检索阶段的 Bi-Encoder 从百万文档中快速筛出约 30 条候选（快但不够准），精排阶段的 Cross-Encoder 再从这 20 多条候选里选出最相关的 5 条（准但慢）。速度和精度的取舍，在检索链路的不同阶段做不同的选择。

3. RerankPostProcessor 代码

精排处理器本身的代码很简短：

```
@Slf4j
@Component
@RequiredArgsConstructor
public class RerankPostProcessor implements SearchResultPostProcessor {

    private final RerankService rerankService;

    @Override
    public String getName() {
        return "Rerank";
    }

    @Override
    public int getOrder() {
        return 10; // 最后执行
    }
}
```

```

@Override
public boolean isEnabled(SearchContext context) {
    return true; // 始终启用
}

@Override
public List<RetrievedChunk> process(List<RetrievedChunk> chunks,
                                    List<SearchChannelResult> results,
                                    SearchContext context) {

    if (chunks.isEmpty()) {
        log.info("Chunk 列表为空, 跳过 Rerank");
        return chunks;
    }

    return rerankService.rerank(
        context.getMainQuestion(),
        chunks,
        context.getTopK()
    );
}
}

```

三个要点：

1. `getOrder() = 10`：在去重（`order=1`）之后执行。去重先把重复清掉，精排再对无重复的候选集统一打分
2. **委托** `RerankService`：处理器本身不关心用哪个 `Rerank` 模型、调哪家 API。它只负责把参数传进去，具体的模型调用交给 `RerankService`——职责分离
3. **一步完成精排 + 截断**：`context.getTopK()` 直接传给 `rerank()` 方法，`Rerank` 服务在打分的同时就做了 `topK` 截断，输出就是最终要喂给大模型的 `Chunk` 列表

还有一个空列表的短路判断：如果去重后一条 `Chunk` 都没有（比如所有通道都失败了），直接返回空列表，不调 `Rerank` API。

4. RoutingRerankService：路由与降级

`RerankPostProcessor` 委托的 `RerankService` 在项目里的默认实现是 `RoutingRerankService`——一个带路由和降级能力的服务：

```

@Service
@Primary
public class RoutingRerankService implements RerankService {

    private final ModelSelector selector;
    private final ModelRoutingExecutor executor;
    private final Map<String, RerankClient> clientsByProvider;

    public RoutingRerankService(ModelSelector selector,
                                ModelRoutingExecutor executor,
                                List<RerankClient> clients) {
        this.selector = selector;
        this.executor = executor;
        this.clientsByProvider = clients.stream()
            .collect(Collectors.toMap(RerankClient::provider, Function.identity()));
    }

    @Override
    public List<RetrievedChunk> rerank(String query, List<RetrievedChunk> candidates, int topN) {
        return executor.executeWithFallback(
            ModelCapability.RERANK,
            selector.selectRerankCandidates(),
            target -> clientsByProvider.get(target.candidate().getProvider()),
            (client, target) -> client.rerank(query, candidates, topN, target)
        );
    }
}

```

```
}  
}
```

这段代码做了三件事：

1. `ModelSelector.selectRerankCandidates()`：从配置中选出可用的 Rerank 模型列表，按优先级排序。比如首选百炼（阿里云），备选 Noop（无操作兜底）
2. `ModelRoutingExecutor.executeWithFallback()`：按优先级依次尝试。首选模型调用成功就返回结果；如果首选模型超时或报错，自动切换到下一个候选
3. `clientsByProvider`：按供应商名称（`provider()`）索引到具体的 `RerankClient` 实现。构造函数通过 Spring 列表注入收集所有 `RerankClient` Bean，然后建立 `provider → client` 的映射

这个设计和第 10 篇多通道检索的容错策略一脉相承：单点故障不影响全局，能降级就降级。

5. BaiLianRerankClient：精排 + 截断的具体实现

`BaiLianRerankClient` 是调用百炼（阿里云）Rerank API 的具体实现。来看核心逻辑。

5.1 入口去重

调用 Rerank API 之前先做一次 ID 去重：

```
@Override  
public List<RetrievedChunk> rerank(String query, List<RetrievedChunk> candidates,  
                                   int topN, ModelTarget target) {  
    if (candidates == null || candidates.isEmpty()) {  
        return List.of();  
    }  
  
    // 入口去重（防止同 ID 文档重复提交给 Rerank API）  
    List<RetrievedChunk> dedup = new ArrayList<>(candidates.size());  
    Set<String> seen = new HashSet<>();  
    for (RetrievedChunk rc : candidates) {  
        if (seen.add(rc.getId())) {  
            dedup.add(rc);  
        }  
    }  
  
    if (topN <= 0 || dedup.size() <= topN) {  
        return dedup;  
    }  
  
    return doRerank(query, dedup, topN, target);  
}
```

这次去重是防御性的——正常情况下上游的 `DeduplicationPostProcessor` 已经去过重了。但万一去重处理器被关闭了（`isEnabled()` 返回 `false`），或者有其他处理器在中间又引入了重复，这里再做一道保险。Rerank API 按文档数量计费，重复文档白花钱。

还有一个提前返回的判断：如果去重后的文档数量已经不超过 `topN`，就不需要调 Rerank API 了，直接返回。比如去重后只剩 3 条而 `topN` 是 5，已经不需要裁剪了。

5.2 结果解析与 topK 截断

`doRerank()` 内部调用百炼 Rerank API 后，对返回结果的解析和截断：

```
List<RetrievedChunk> reranked = new ArrayList<>();  
Set<String> addedIds = new HashSet<>();  
  
for (JsonElement elem : results) {  
    // ... 解析 index 和 relevance_score ...  
    RetrievedChunk src = candidates.get(idx);  
  
    Float score = null;  
    if (item.has("relevance_score") && !item.get("relevance_score").isJsonNull()) {  
        score = item.get("relevance_score").getAsFloat();  
    }  
}
```



```
}

RetrievedChunk hit = score != null
    ? new RetrievedChunk(src.getId(), src.getText(), score)
    : src;
reranked.add(hit);
addedIds.add(src.getId());

if (reranked.size() >= topN) {
    break; // 达到 topN 就停
}
}

// 如果 Rerank API 返回的结果不足 topN, 用原始候选补齐
if (reranked.size() < topN) {
    for (RetrievedChunk c : candidates) {
        if (addedIds.add(c.getId())) {
            reranked.add(c);
        }
        if (reranked.size() >= topN) {
            break;
        }
    }
}
}
```

三个关键设计：

relevance_score 覆写原始分数：Rerank API 返回的 `relevance_score` 是 Cross-Encoder 对 query-chunk 对的统一相关性打分（通常在 0~1 之间）。不管这条 Chunk 原来的分数是余弦相似度 0.82 还是 BM25 分数 12.5，经过 Rerank 后都变成了同一标准下的相关性评分。这就解决了前面提到的分数不可比问题。

边解析边截断：`reranked.size() >= topN` 时直接 `break`，不需要把 API 返回的所有结果都解析完再截断。Rerank API 本身返回的结果已经按相关性从高到低排好了，前 `topN` 条就是最相关的。

不足 topN 时用原始候选补齐：如果 Rerank API 返回的结果不够 `topN`（比如 API 只返回了 3 条但 `topN` 是 5），用原始候选列表中未被选中的 Chunk 按原始顺序补齐。这个补齐策略保证输出数量尽量达标，不会因为 API 少返回了几条就让下游拿到不够数的结果。

5.3 NoopRerankClient：终极兜底

当所有 Rerank 模型都不可用时（首选百炼超时，备选也挂了），`RoutingRerankService` 会降级到 `NoopRerankClient`：

```
@Service
public class NoopRerankClient implements RerankClient {

    @Override
    public String provider() {
        return ModelProvider.NOOP.getId();
    }

    @Override
    public List<RetrievedChunk> rerank(String query, List<RetrievedChunk> candidates,
                                       int topN, ModelTarget target) {
        if (candidates == null || candidates.isEmpty()) {
            return List.of();
        }
        if (topN <= 0 || candidates.size() <= topN) {
            return candidates;
        }
        return candidates.stream()
            .limit(topN)
            .collect(Collectors.toList());
    }
}
```

逻辑极其简单：不做精排，直接按原始顺序截断前 `topN` 条。效果当然不如 Cross-Encoder 精排——排在前面的不一定是最相关的。但至少保证了流水线不中断，有结果总比没结果好。

链式编排：executePostProcessors

1. Spring 自动注入 + 排序

处理器的注册和编排在 MultiChannelRetrievalEngine 的 executePostProcessors() 方法中完成：

```
private List<RetrievedChunk> executePostProcessors(List<SearchChannelResult> results,
                                                    SearchContext context) {

    // 过滤启用的处理器并排序
    List<SearchResultPostProcessor> enabledProcessors = postProcessors.stream()
        .filter(processor -> processor.isEnabled(context))
        .sorted(Comparator.comparingInt(SearchResultPostProcessor::getOrder))
        .toList();

    if (enabledProcessors.isEmpty()) {
        log.warn("没有启用的后置处理器，直接返回原始结果");
        return results.stream()
            .flatMap(r -> r.getChunks().stream())
            .collect(Collectors.toList());
    }

    // 初始 Chunk 列表（所有通道的结果合并）
    List<RetrievedChunk> chunks = results.stream()
        .flatMap(r -> r.getChunks().stream())
        .collect(Collectors.toList());

    int initialSize = chunks.size();

    // 依次执行处理器
    for (SearchResultPostProcessor processor : enabledProcessors) {
        try {
            int beforeSize = chunks.size();
            chunks = processor.process(chunks, results, context);
            int afterSize = chunks.size();

            log.info("后置处理器 {} 完成 - 输入: {} 个 Chunk, 输出: {} 个 Chunk, 变化: {}",
                processor.getName(),
                beforeSize,
                afterSize,
                (afterSize - beforeSize > 0 ? "+" : "") + (afterSize - beforeSize)
            );
        } catch (Exception e) {
            log.error("后置处理器 {} 执行失败, 跳过该处理器", processor.getName(), e);
        }
    }

    log.info("后置处理器链执行完成 - 初始: {} 个 Chunk, 最终: {} 个 Chunk",
        initialSize, chunks.size());

    return chunks;
}
```

List<SearchResultPostProcessor> postProcessors 是通过 Spring 构造器注入的。Spring 会自动收集所有实现了 SearchResultPostProcessor 接口并标注了 @Component 的 Bean，注入到这个列表中。然后按 getOrder() 升序排列——当前排列结果是 Deduplication (order=1) → Rerank (order=10)。

2. 链式传递机制

核心就是 for 循环里的这行：

```
chunks = processor.process(chunks, results, context);
```

上一个处理器的输出 `chunks` 直接变成下一个处理器的输入——经典的职责链模式。每个处理器只管自己的事：去重处理器只负责去掉重复，精排处理器只负责重打分和截断。谁先谁后由 `getOrder()` 决定，处理器之间不需要互相感知。

3. 每步日志与漏斗监控

每个处理器执行完后都会打一条日志，记录输入输出的 `Chunk` 数量变化。实际运行时的日志长这样：

```
后置处理器 Deduplication 完成 - 输入：28 个 Chunk，输出：21 个 Chunk，变化：-7
后置处理器 Rerank 完成 - 输入：21 个 Chunk，输出：5 个 Chunk，变化：-16
后置处理器链执行完成 - 初始：28 个 Chunk，最终：5 个 Chunk
```

运维看到这三行日志就能清楚知道：漏斗在哪一步收窄、每步丢了多少。如果某天发现去重步骤从 `-7` 变成了 `-0`，说明通道之间完全没有重复，可能意味着通道配置变了或者某个通道没启用。如果精排步骤从 `-16` 变成了 `-0`，很可能 `Rerank` 模型出了问题，降级到了 `NoopRerankClient`。

4. 单处理器异常不中断链

`try-catch` 包裹了每个处理器的执行：

```
    } catch (Exception e) {
        log.error("后置处理器 {} 执行失败，跳过该处理器", processor.getName(), e);
        // 继续执行下一个处理器，不中断整个链
    }
```

如果 `Rerank` API 超时了，精排处理器抛异常被 `catch` 住，`chunks` 保持去重后的 21 条不变，继续往下走。21 条虽然比精选的 5 条多了很多，但至少不是完全没结果。这个设计和第 10 篇多通道检索的容错策略一脉相承：单点故障降级处理，不让一个环节的异常拖垮整个链路。

5. `getOrder()` 的间隔设计

去重的 `order` 是 1，精排的 `order` 是 10。中间隔了 2~9 共八个位置，不是随意选的。如果未来需要在去重和精排之间插入新处理器，比如 `order=3` 的时效性过滤（过滤掉半年前的过期文档），或者 `order=5` 的权限过滤（过滤掉用户无权访问的文档），直接加就行，不需要改动已有处理器的 `order` 值。这是一种常见的工程实践——给优先级留间隔。类似 CSS 的 `z-index` 习惯用 10、20、30 而不是 1、2、3，数据库的自增步长设成 10 而不是 1——都是为了未来插入新元素时不需要改动已有元素。

6. 为什么去重和精排分成两个处理器

把去重和精排写成一个处理器当然也能跑通——先去重再精排，逻辑一样。但拆成两个处理器有三个好处：

- 1. **单一职责**：去重处理器只管去重，精排处理器只管精排。各自的逻辑清晰，测试也简单——测去重不需要 `Mock Rerank` 模型，测精排不需要准备重复数据
- 2. **独立开关**：如果某个场景不需要精排（比如只有一个通道启用，不存在分数不可比的问题），可以通过 `isEnabled()` 单独关掉精排处理器，去重继续生效
- 3. **独立替换**：如果要把 `Cross-Encoder` 精排换成其他重排算法（比如基于规则的重排），只需要写一个新的处理器替换 `RerankPostProcessor`，不影响去重逻辑

扩展性：怎么加一个新处理器

假设产品提了一个需求：过滤掉半年前更新的过期文档，只保留近期内容。怎么做？

```
@Component
public class FreshnessFilterPostProcessor implements SearchResultPostProcessor {

    @Override
    public String getName() {
        return "FreshnessFilter";
    }

    @Override
    public int getOrder() {
```

```
        return 5; // 在去重（1）之后、精排（10）之前
    }

    @Override
    public boolean isEnabled(SearchContext context) {
        return true;
    }

    @Override
    public List<RetrievedChunk> process(List<RetrievedChunk> chunks,
                                       List<SearchChannelResult> results,
                                       SearchContext context) {
        LocalDate sixMonthsAgo = LocalDate.now().minusMonths(6);
        return chunks.stream()
            .filter(chunk -> {
                // 从元数据中获取更新时间，过滤掉过期文档
                // 没有更新时间的默认保留
                return true; // 具体过滤逻辑根据 Chunk 元数据实现
            })
            .collect(Collectors.toList());
    }
}
```

做完了。不需要改 MultiChannelRetrievalEngine 的一行代码，不需要改 DeduplicationPostProcessor 或 RerankPostProcessor 的一行代码。Spring 容器启动时会自动发现这个新 Bean，注入到 postProcessors 列表中，按 order=5 排在去重和精排之间。

这就是 Spring 自动注入 + 接口编程的威力：新增能力只需要写新代码，不需要改旧代码。对应到设计原则就是开闭原则（Open-Closed Principle）——对扩展开放，对修改关闭。简单点说，这个也是策略模式的具体实现。

流水线加上新处理器后的执行顺序变成了：

```
Deduplication (order=1) → FreshnessFilter (order=5) → Rerank (order=10)
```

漏斗也多了一层：28 → 去重 21 → 时效过滤 18 → 精排 5。

KB 检索链路完整串联

到这里，后处理流水线讲完了。回过头来看 KB 检索的完整链路——从用户的一句话，到最终喂给大模型的 5 条高质量上下文，一共经过了四个阶段：

篇号	主题	做了什么	数据变化
5~8	意图识别	判断用户想查哪个知识库，打 分、筛选、歧义引导	用户问题 → 意图列表
9	意图到检索的映射	把意图转化成具体的检索参 数：查哪个 Collection、查多 少条	意图 → Collection + topK
10	多通道并行检索	三个通道同时去各个库里捞文 档，并行调度、容错	检索参数 → ~30 条原始 Chunk
11（本篇）	后处理流水线	去重 + 精排 + 截断	~30 条 → 5 条高质量 Chunk

用户问“AirPods Pro 怎么退货？”这个例子串一遍：意图识别命中 3C 数码退货政策 → 定向检索通道查 kb_3c_return Collection，全局向量通道做兜底检索 → 两个通道并行跑完，返回 28 条 Chunk → 去重合并为 21 条 → Cross-Encoder 精排，按统一的相关性评分从高到低排序，截断到 5 条 → 这 5 条 Chunk 作为上下文喂给大模型生成答案。

到这里，KB 检索链路就完整收尾了。

小结与下一篇预告

本篇拆解了多通道检索的后处理流水线，核心要点：

1. 多通道检索的原始结果有三个问题——重复、分数不可比、数量太多。后处理流水线是把粗糙的多源结果变成精炼的高质量上下文的关键环节
2. `SearchResultPostProcessor` 接口定义了四个方法，`process()` 同时接收流水线传递对象（`chunks`）和原始通道结果（`results`），满足不同处理器的不同数据需求
3. `DeduplicationPostProcessor`（`order=1`）做去重：按通道优先级处理，`LinkedHashMap` 保序去重，ID 优先 + 内容哈希兜底，重复时保留高分
4. `RerankPostProcessor`（`order=10`）做精排 + 截断：Cross-Encoder 用统一的 `relevance_score` 解决分数不可比问题，同时完成 topK 截断
5. `RoutingRerankService` 通过模型路由 + 降级保证 Rerank 的高可用，`NoopRerankClient` 作为终极兜底——Rerank 全挂了就按原始顺序截断
6. `executePostProcessors()` 用 Spring 自动注入 + `getOrder()` 排序实现链式编排，单处理器异常不中断链，每步日志记录漏斗变化
7. 整条流水线的漏斗效果：约 30 条 → 去重约 20 条 → 精排 + 截断 5 条
8. 扩展性良好：新增处理器只需实现接口 + `@Component` + 设置 `order`，零侵入已有代码

到这里 KB 检索链路就完整了——意图识别告诉系统该查哪个知识库，多通道检索把文档捞出来，后处理流水线把结果精炼到 5 条。但 KB 检索不是万能的。用户问“我的订单到哪了？”，知识库里没有这个用户的实时订单数据，靠检索文档回答不了。第 10 篇提到过 MCP 意图和 KB 意图是并行处理的，那 MCP 工具调用链路是怎么跑的？工具定义怎么注册？参数怎么传？调用结果怎么拼回大模型的上下文？第 12 篇讲——知识库答不了的问题，交给 MCP 工具去查。

